

CoAP Protocol

Constrained Application Protocol

Panwit Tuwanut, IT KMITL

CoAP คืออะไร

- CoAP ย่อมาจาก Constrained Application Protocol
- เป็นโปรโตคอลสื่อสารระดับแอปพลิเคชันที่ออกแบบมาโดยเฉพาะสำหรับ
  - อุปกรณ์ IoT (Internet of Things)
  - เครือข่ายที่มีทรัพยากรจำกัด (low-power / low-memory)
- สื่อสารผ่าน UDP บนเครือข่าย IPv4/IPv6
- CoAP ถูกกำหนดโดย RFC 7252 โดย IETF และมีลักษณะคล้ายกับ HTTP แต่เบากว่าและเหมาะกับไมโครคอนโทรลเลอร์ เช่น Arduino หรือ ESP8266, ESP32

CoAP คืออะไร

- RFC 7252 defines the core of the CoAP protocol. It was designed for machine-to-machine (M2M) applications such as smart energy and building automation, supporting constrained devices and networks while cooperating with HTTP through simple proxies.
- CoAP reuses the web concepts of URIs and Internet media types to enable the building of IoT solutions.

How CoAP Works (Architecture Overview)

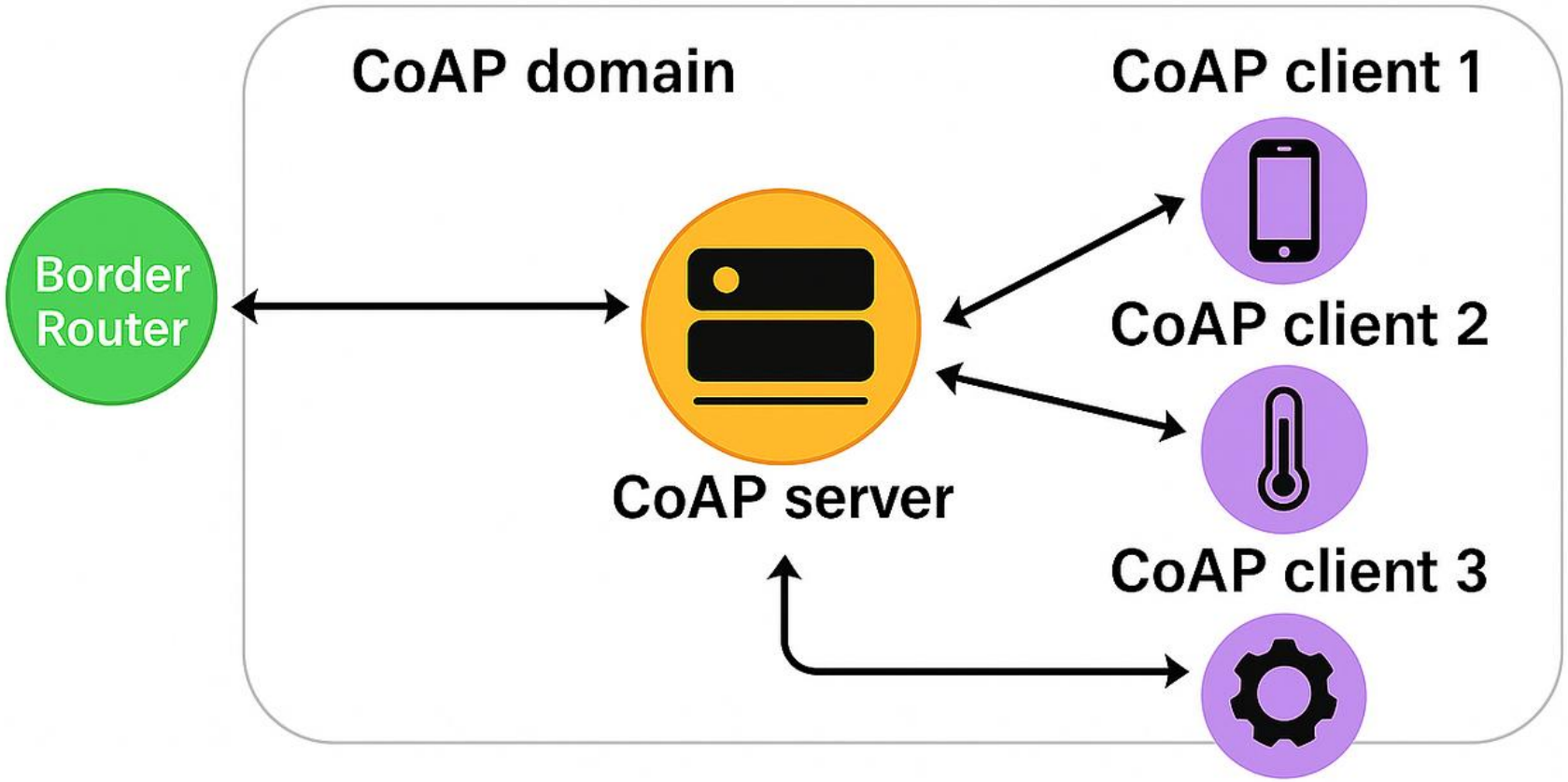
CoAP uses a simple **request/response model** similar to HTTP. Here's how it functions:

1. **Client sends a request** (e.g., GET a temperature reading).
2. **Server replies** with the sensor data.
3. **Requests and responses** use a binary message format with options encoded in a Type-Length-Value (TLV) format.
4. Optional **tokens** are used to match requests with responses.
5. **Reliability** is achieved through retransmissions and acknowledgments.

Features & Functions of CoAP

Feature	Description
RESTful API	Uses GET, PUT, POST, DELETE methods like HTTP
Compact Messages	Small binary headers (4 bytes minimum)
Multicast Support	Efficient for sending the same command to multiple devices
Observability	Clients can “observe” resources and get updates when data changes
Low Memory Use	Can run on devices with just 10 KiB RAM
DTLS Security	Provides robust security comparable to 3072-bit RSA
Efficient Caching	Supports freshness indicators for caching responses
Proxy Support	Seamlessly connects with web apps via CoAP-HTTP proxies

CoAP Protocol



CoAP Server : ผู้ให้บริการ Resource

Resource : สิ่งที่ให้บริการ

URI/Path : ที่อยู่ของ Resource

CoAP Client : ผู้เข้ามาใช้บริการ

หลักการทำงานของ CoAP

- CoAP เป็นโปรโตคอลแบบ Client–Server คล้ายกับโปรโตคอล HTTP
  - **Client** ส่งคำขอ (Request) ไปยัง **Server** และ Server ตอบกลับ (Response)
    - Server จะเป็นฝั่งที่มี Resource เช่น sensor
    - Client จะเป็นฝั่งร้องขอข้อมูล
- ใช้หลักการ **RESTful API**: มี GET, POST, PUT, DELETE
- อ้างอิงด้วย **URI** เช่น coap://192.168.1.10/led

- CoAP Server เป็นฝั่งที่ให้บริการ Resource
  - **Resource** ใน CoAP คือ “สิ่งที่ Server เปิดให้ Client เข้ามาใช้งานผ่าน URI”

- ข้อสังเกต
  - MQTT เป็นโปรโตคอลแบบ Publish/Subscribe

หลักการทำงานของ CoAP

ทั้งนี้ อุปกรณ์ตัวเดียว สามารถเป็นได้ทั้ง Client และ Server ได้

เช่น Raspberry Pi มี Sensor ของตัวเองและเปิด CoAP Server เพื่อให้อุปกรณ์อื่นมาอ่านข้อมูล

ขณะเดียวกันก็เป็น CoAP Client เพื่อไปอ่านข้อมูลจากอุปกรณ์ตัวอื่น

ดังนั้น Sensor/Actuator สามารถอยู่ได้ทั้งฝั่ง Client หรือฝั่ง Server ก็ได้ ขึ้นอยู่กับรูปแบบการสื่อสาร

CoAP (RESTful)

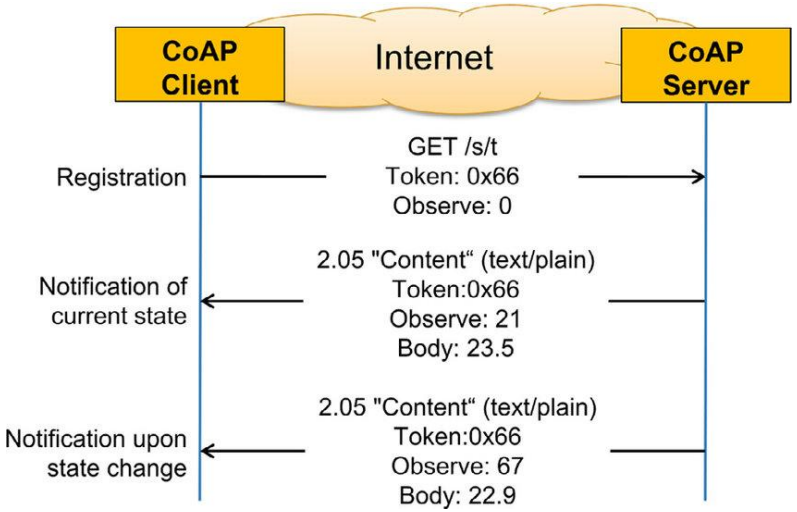
CoAP Method จะใช้เหมือน HTTP Method

เมธอด	ความหมาย	ตัวอย่าง
GET	ขอข้อมูลจาก Resource เช่น อ่าน Temperature	/sensor/temp
POST	สร้าง/ส่ง ข้อมูลใหม่ เช่น เพิ่ม Device	/device/register
PUT	แก้ไข/กำหนด ค่าข้อมูล เช่น เปิด/ปิด LED	/led/state
DELETE	ลบข้อมูล เช่น ลบ Resource	/config

Feature พิเศษ: Observe

CoAP Observe ทำให้ Client สามารถรับ Notification จาก Resource ได้ เมื่อข้อมูลเปลี่ยนแปลง

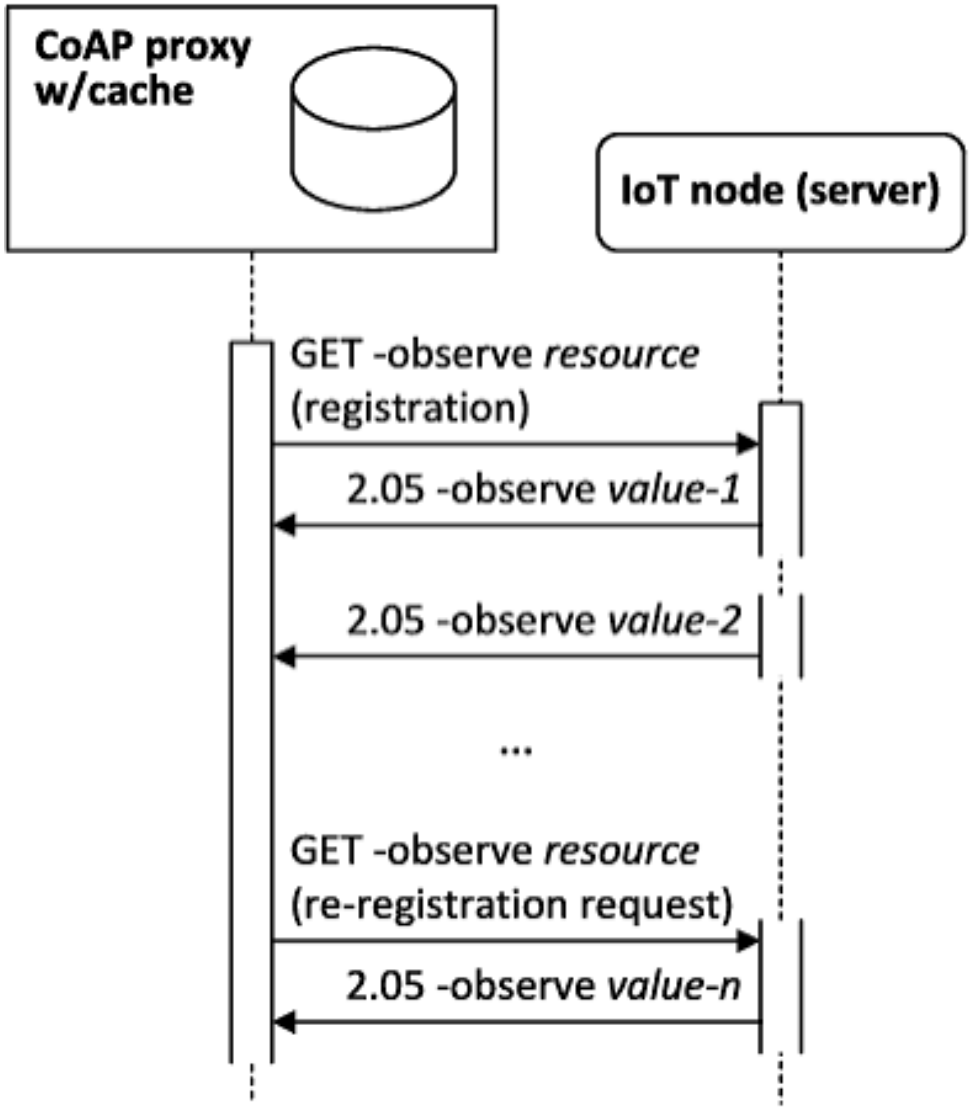
- Client สามารถ "สังเกตการณ์" (Observe) resource ได้
- เมื่อ resource เปลี่ยนค่า จะส่งผลให้ Server จะทำการ **push ข้อมูลใหม่ให้อัตโนมัติ**
- ใช้ดีกับ **sensor ที่มีการเปลี่ยนแปลง สถานะ หรือข้อมูล real-time**



The observe extension allows clients to “subscribe” to a server URI, so that notifications are sent whenever the state represented by that URI changes

Non-confirmable messages have no acknowledgement, similar to QoS 0 in MQTT.

Feature พิเศษ: Observe




การทดลอง
Raspberry Pi &
CoAP protocol

Library : aiocoap

- aiocoap เป็น library ของภาษา python สำหรับเขียนโปรแกรม CoAP แบบ Asynchronous โดยที่

  aio ย่อมาจาก Asynchronous I/O

  coap ย่อมาจาก Constrained Application Protocol

Method ฝั่ง Server

Method ของ aiocoap	หน้าที่
`Context.create_server_context()`	เริ่มต้น CoAP Server
`root.add_resource()`	สร้าง Resource
`render_get(self, request)`	จัดการ GET Request `request.payload` ข้อมูลที่ Client ส่งมา เป็น bytes `request.code` Method ที่ส่งมา เช่น GET, PUT, POST `request.remote` ข้อมูลของ Client ที่เชื่อมต่อมา
`render_post(self, request)`	จัดการ POST Request
`render_put(self, request)`	จัดการ PUT Request
`render_delete(self, request)`	จัดการ DELETE Request
`Message(code = , payload = )`	สร้าง CoAP Response กลับไปยัง Client
`self.updated_state()`	แจ้งว่าข้อมูลเปลี่ยนแปลง เพื่อส่ง Observe Notification เป็น Option ของ GET

Response code ของ CoAP Server : Success

Code	ความหมาย	ตัวอย่าง
2.01 Created	สร้าง Resource สำเร็จ	POST /device
2.02 Deleted	ลบ Resource สำเร็จ	DELETE /device
2.03 Valid	Resource ยัง valid / ใช้กับ Cache validation	ตรวจสอบข้อมูลเดิม
2.04 Changed	เปลี่ยน Resource สำเร็จ	PUT /led
2.05 Content	ส่งข้อมูลกลับสำเร็จ	GET /temperature

Response code ของ CoAP Server : Client Error

Code	ความหมาย
4.00 Bad Request	Request ไม่ถูกต้อง
4.01 Unauthorized	ต้องยืนยันตัวตน
4.02 Bad Option	Option ไม่ถูกต้อง
4.03 Forbidden	ไม่มีสิทธิ์
4.04 Not Found	ไม่พบ Resource
4.05 Method Not Allowed	Method นี้ใช้กับ Resource นี้ไม่ได้
4.06 Not Acceptable	ไม่สามารถตอบในรูปแบบที่ร้องขอ
4.08 Request Entity Incomplete	Request ยังไม่ครบ
4.12 Precondition Failed	เงื่อนไขก่อนทำงานไม่ผ่าน
4.13 Request Entity Too Large	ข้อมูลที่ส่งใหญ่เกินไป

Response code ของ CoAP Server : Server Error

Code	ความหมาย
5.00 Internal Server Error	Server เกิดข้อผิดพลาด
5.01 Not Implemented	Server ไม่รองรับ
5.02 Bad Gateway	Gateway ได้ response ที่ผิด
5.03 Service Unavailable	Server ไม่พร้อมให้บริการ
5.04 Gateway Timeout	Gateway รอ Server นานเกินไป
5.05 Proxying Not Supported	ไม่รองรับการ Proxy

ตัวอย่าง code ส่วน CoAP Server

- สร้าง Resource เช่น

```
root.add_resource(
    ["temperature"],
    TemperatureResource()
)
```

หมายถึงได้ URI เป็น
coap://127.0.0.1:5683/temperature

```
root.add_resource(
    ["led"],
    LEDResource()
)
```

หมายถึงได้ URI เป็น
coap://127.0.0.1:5683/led

ตัวอย่าง code ส่วน CoAP Server

สร้าง Resource สำหรับจัดการข้อมูล Temperature

โดยจะต้องระบุว่า ใช้ method อะไร (GET, PUT,..)

ทำการ ประมวลผล GET Request และสร้าง Response กลับไปให้ Client

```
class TemperatureResource(resource.Resource):
    async def render_get(self, request):
        temperature = "25.6"
        print("GET /temperature")

        return Message(
            code=aiocoap.Code.CONTENT,
            payload=temperature.encode()
        )
```

ตัวอย่าง code ส่วน CoAP Server

สร้าง Resource สำหรับจัดการข้อมูล LED

โดยจะต้องระบุว่า ใช้ method อะไร (GET, PUT,..)

ทำการ ประมวลผล PUT Request และสร้าง Response กลับไปให้ Client

```
class LEDResource(resource.Resource):
    async def render_put(self, request):
        command = request.payload.decode()
        print("PUT /led =", command)
        if command == "ON":
            GPIO.output(LIGHT, True)
            print("LED ON")
        return Message(
            code=aiocoap.Code.CHANGED,
            payload=b"OK"
        )
```

Method ฝั่ง Client

Method ของ aiocoap	หน้าที่
`Context.create_client_context()`	สร้าง CoAP Client Context ซึ่งเป็นตัวจัดการการสื่อสาร CoAP
`Message(code=Code.GET, uri=...)`	ส่ง GET Request
`Message(code=Code.POST, uri=..., payload=...)`	ส่ง POST Request
`Message(code=Code.PUT, uri=..., payload=...)`	ส่ง PUT Request
`Message(code=Code.DELETE, uri=...)`	ส่ง DELETE Request
`await protocol.request(request).response`	รับ Response จาก Server
`request.opt.observe = 0`	สมัครการรับข้อมูลแบบ Observe Option เป็น Option ของ GET

ตัวอย่าง code ส่วน CoAP Client

สร้าง GET Request

```
async def get_temperature():

    protocol = await Context.create_client_context()
    request = Message(
        code=aiocoap.Code.GET,
        uri=f"coap://192.168.1.164:5683/temperature"
    )

    response = await protocol.request(request).response
```

Note!!

protocol.request(request) คือการ ส่ง Request ออกไปยัง CoAP Server

.response คือ รอรับ Response จาก Server

ตัวอย่าง code ส่วน CoAP Client

สร้าง PUT Request

```
async def led_on():

    protocol = await Context.create_client_context()
    request = Message(
        code=aiocoap.Code.PUT,
        uri=f"coap://192.168.1.164:5683/led",
        payload=b"ON"     #ส่งเป็น byte
    )

    response = await protocol.request(request).response
```

Note!!

protocol.request(request) คือการ ส่ง Request ออกไปยัง CoAP Server

.response คือ รอรับ Response จาก Server

การทดลอง. ติดตั้ง Library aiocoap

```
$ sudo apt update
$ sudo apt install python3-pip
```

สร้าง virtual environment:

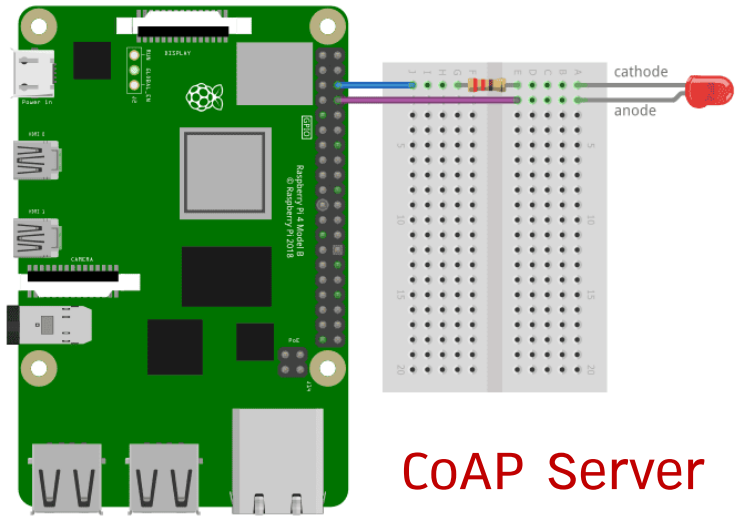
```
$ sudo apt install python3-full python3-venv –y
$  python3 -m venv ~/coap-env
$  source ~/coap-env/bin/activate
```

ติดตั้ง Library

```
(coap-env) $ pip3 install aiocoap
(coap-env) $ pip3 install RPi.GPIO
(coap-env) $ pip3 install --upgrade websockets
```

การทดลอง. Coap with Raspberry Pi

- นำมือถือของ นศ เปิด Hot Spot ตั้งค่า SSID, Password
- แก้ค่า IP Address ในโปรแกรม ให้ถูกต้อง
- ฝั่ง CoAP Server ต่อ LED เข้าที่ GPIO 14
- ทำการ Run Code ที่แก้ไขแล้ว
- สังเกตผลการทดลอง



CoAP Server



CoAP Client

Code ตัวอย่าง coap Server.py , coap Client.py

แก้ Code ในส่วนต่อไปนี้

coap server.py

แก้ IP Address ให้เป็นของตนเอง โดยใช้คำสั่ง ifconfig เพื่อดู IP Address ของตนเอง

```
await Context.create_server_context(root , bind=("192.168.1.100", 5683))
```

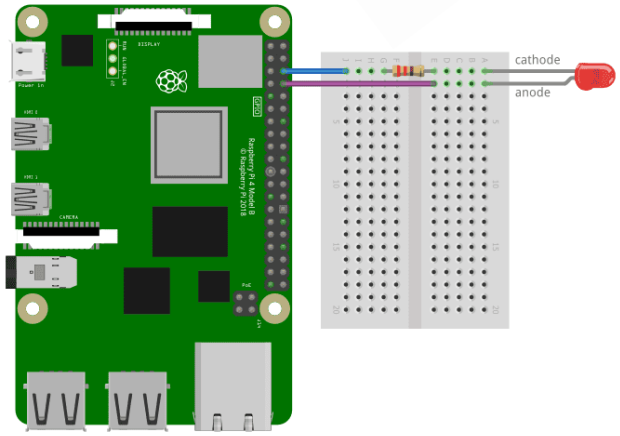
coap client.py

แก้ IP Address ให้เป็น IP ของ Coap Server

```
SERVER_IP = "192.168.1.100"
```

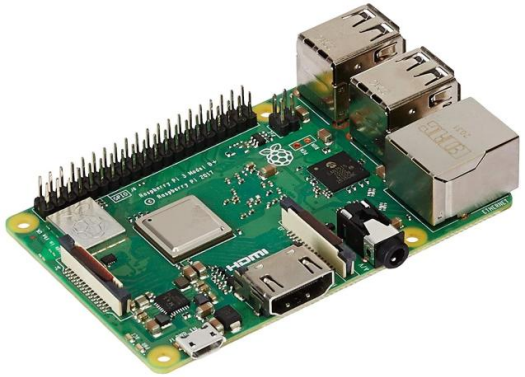
การทดลอง : GET

CoAP Server





CoAP Client



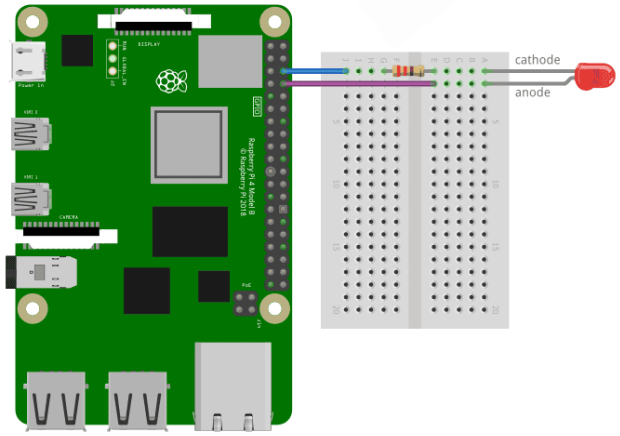
```
request = Message(
        code=aiocoap.Code.GET,
        uri=f"coap://192.168.1.100:5683/temperature"
        )
```

```
temperature = "25.6"
return Message(
           code=aiocoap.Code.CONTENT,
           payload=temperature.encode()
          )
```

- CoAP Server สร้าง resource ชื่อ temperature
- CoAP Client ทำการ GET request ไปที่ CoAP server
- CoAP Server จะ response ด้วยค่า อุณหภูมิ

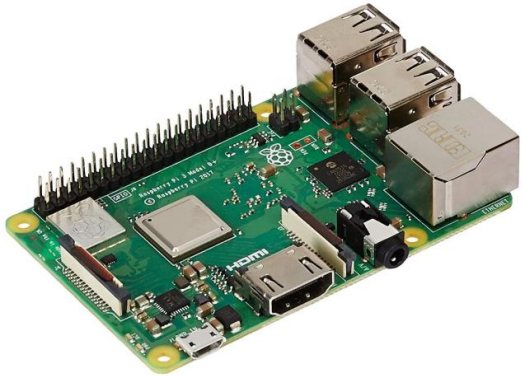
การทดลอง : PUT

CoAP Server





CoAP Client



```
request = Message(
        code=aiocoap.Code.PUT,
        uri=f"coap://192.168.1.100:5683/led",
         payload=b"ON"
        )
```

```
command = request.payload.decode()
        if command == "ON":
            GPIO.output(LIGHT, True)
        elif command == "OFF":
            GPIO.output(LIGHT, False)
```

- CoAP Server สร้าง resource ชื่อ led
- CoAP Client ทำการ PUT request ไปที่ CoAP server
- CoAP Server จะสั่งให้ LED ติด/ดับ ตาม payload ที่รับมา

แบบฝึกหัดที่ 1. PUT

- ให้ CoAP Client ทำการ PUT Request ส่งค่าใดๆ ในช่วง 0-100

ไปยัง CoAP Server แล้วให้ LED มีค่าความสว่างมากน้อย ตามค่าที่ได้รับมา

- Hint : ใช้ เรื่อง PWM ปรับค่า Duty cycle เพื่อปรับความสว่าง



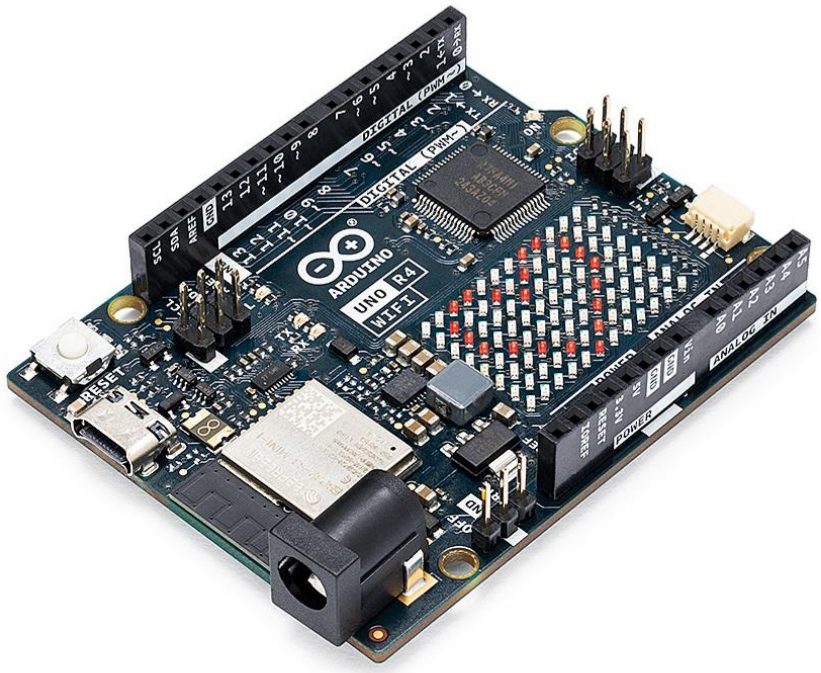
การทดลอง

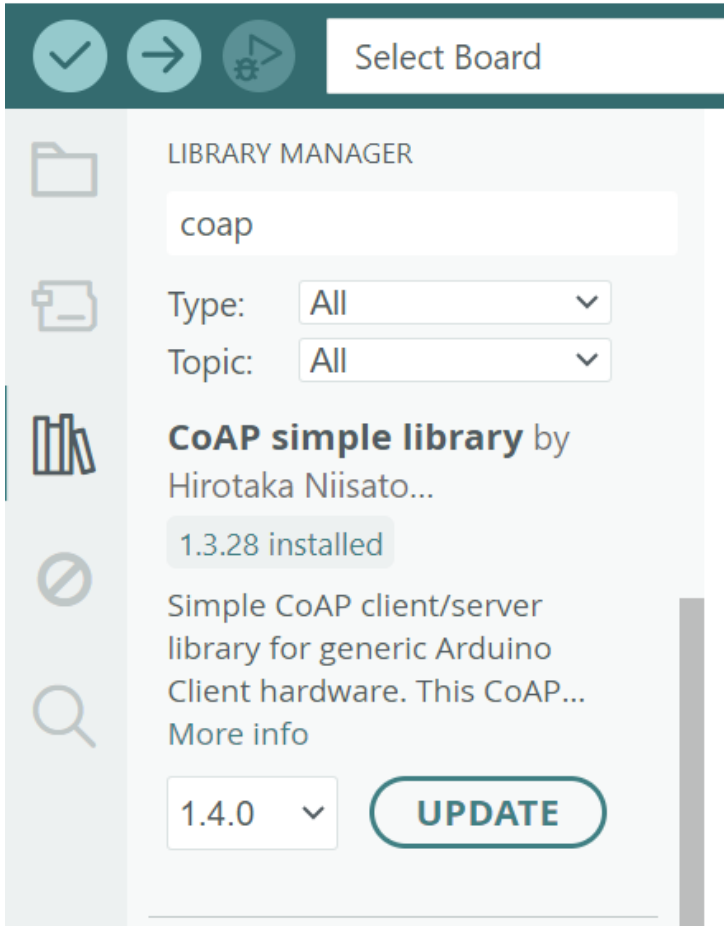
Arduino & CoAP protocol

Laboratory : Library coap-simple

ติดตั้ง Library

ค้นหา Coap จากนั้นกด **INSTALL**





https://github.com/hirotakaster/CoAP-simple-library

Library : coap-simple.h

- Method ฝั่ง Server
  - `coap.start()` เริ่มต้น CoAP server
  - `coap.loop()` ใช้รันและประมวลผล packet

  - `coap.server(callback, "resource")` ใช้ สร้าง resource
    - `callback` ฟังก์ชันที่จะถูกเรียกเมื่อมี CoAP request เข้ามา
    - `resource` ชื่อ resource (หรือ URI path)

  รูปแบบ callback

  - `void callback(CoapPacket &packet, IPAddress ip, int port)`
    - `packet` ข้อมูล CoAP packet
    - `ip` IP ของ client
    - `port` port ของ client

```
packet.code          // Method (GET, POST, PUT, DELETE)
packet.payload       // ข้อมูลที่ส่งมา
packet.payloadlen    // ความยาว payload
packet.messageid     // Message ID
```

  - `coap.sendResponse(ip, port, messageid, payload)`
    ใช้ส่ง response กลับไปยัง client

  - `coap.notify("resource")` ใช้คู่กับ coap.observe ใช้ส่งข้อมูล เมื่อมีการเปลี่ยนแปลงค่า

Library : coap-simple.h

- Method ฝั่ง Client
  - `coap.get(ip, port, "resource")` get request
  - `coap.post(ip, port, "resource", payload, length)` post request
  - `coap.put(ip, port, "resource", payload, length)` put request
  - `coap.delete(ip, port, "resource")` delete request

  - `coap.response(responseCallback)` ตั้ง callback สำหรับ รับ response จาก server

  รูปแบบ callback
  - `void responseCallback(CoapPacket &packet, IPAddress ip, int port)`
    - `packet` ข้อมูล CoAP packet
    - `ip` IP ของ client
    - `port` port ของ client

  - `coap.observe(ip, port, "resource");` ใช้สมัครรับข้อมูลแบบ Observe , ใช้กับ method get เท่านั้น
  - `coap.cancelObserve()` ยกเลิก observe

Code ตัวอย่าง Coap Server.ino , Coap Client.ino

แก้ Code ในส่วนนี้

Coap Server.ino

```
// ===== ใส่ WiFi SSID, Password ของตัวเอง =====
const char WIFI_SSID[] = "panwit_2.4G";     // CHANGE TO YOUR WIFI SSID
const char WIFI_PASSWORD[] = "ilovekmitl";  // CHANGE TO YOUR WIFI PASSWORD
```

Coap Client.ino

```
// ===== ใส่ WiFi SSID, Password ของตัวเอง =====
const char WIFI_SSID[] = "panwit_2.4G";     // CHANGE TO YOUR WIFI SSID
const char WIFI_PASSWORD[] = "ilovekmitl";  // CHANGE TO YOUR WIFI PASSWORD

// กำหนด IP ของ CoAP Server (จากตัวอย่างที่ 1)
IPAddress serverIP(192, 168, 1, 123);   // <-- แก้ IP ให้ตรงกับ IP ของ CoAP Server
```

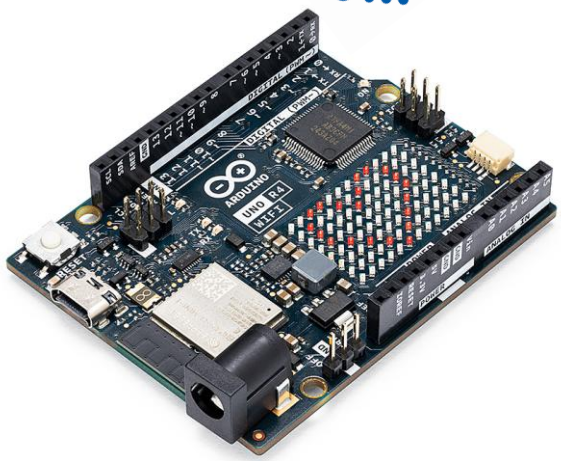
การทดลอง GET

Note !!

ให้ Run Server ก่อน เพื่อดู IP Address โดยดูจากคำสั่ง

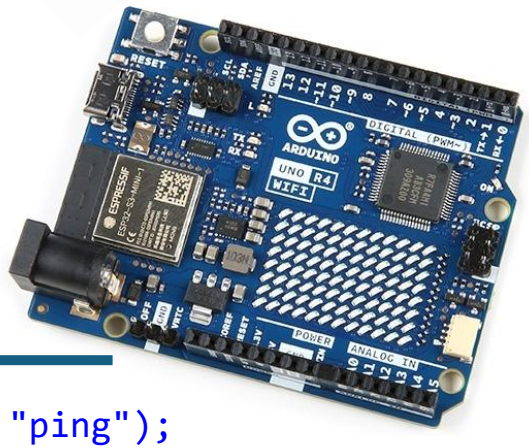
```
Serial.println(WiFi.localIP());
```

Coap Server





Coap Client



```
coap.get(serverIP, COAP_PORT, "ping");
```

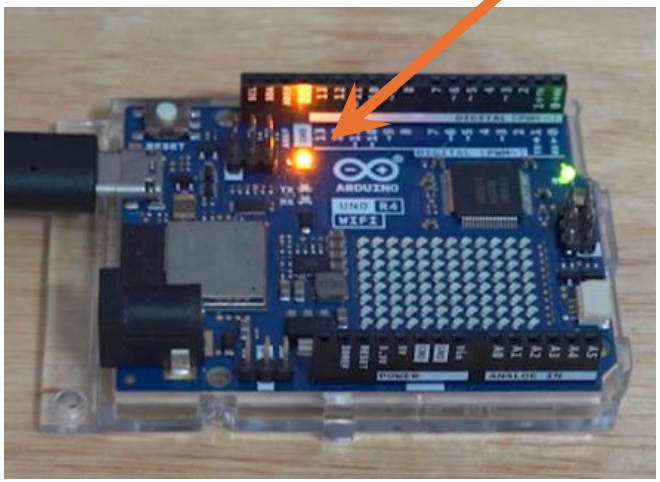
```
const char* msg = "pong";
coap.sendResponse(ip, port, packet.messageid, msg);
```

- Coap Server สร้าง resource ชื่อ ping
- Coap Client ทำการ get request ไปที่ Coap server
- Coap Server จะ response ว่า pong

การทดลอง PUT

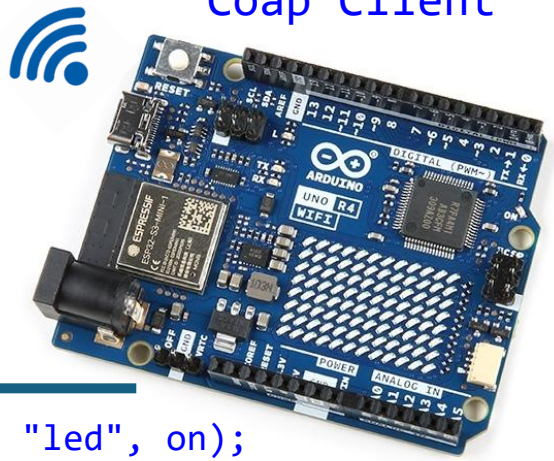
- Coap Server สร้าง resource ชื่อ led
- Coap Client ทำการ put ส่งค่า on/off สลับกันทุกๆ 5 วินาที
- Coap Server สั่งให้ Built in LED ติดดับ ตามค่า on/off ที่รับมา

Coap Server



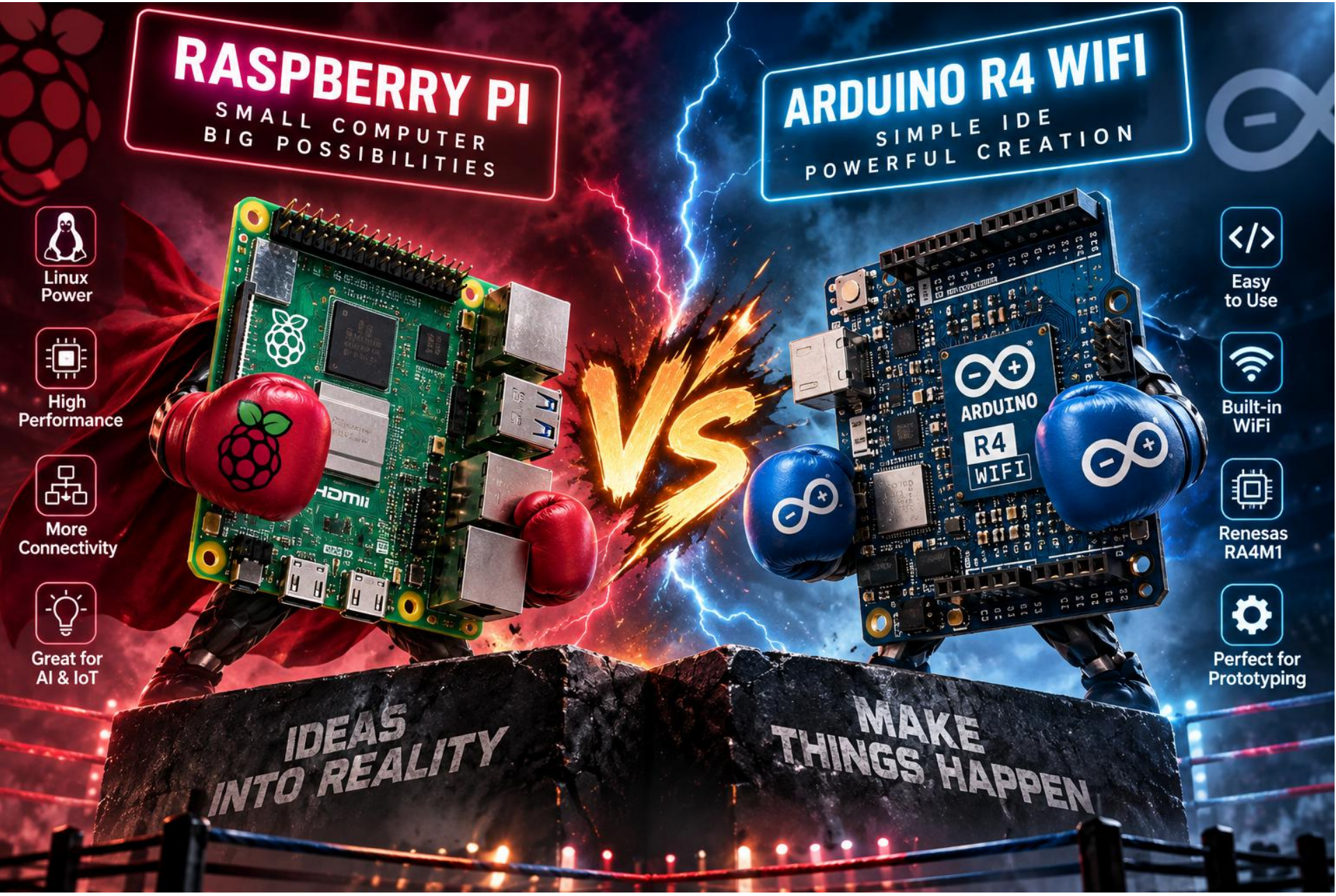


Coap Client



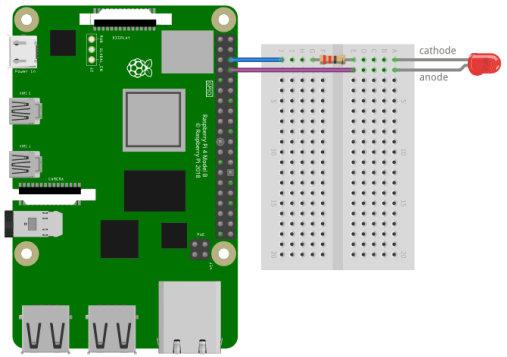
```
coap.put(serverIP, 5683, "led", on);
```

```
if (payload.equalsIgnoreCase("on")) {
   digitalWrite(LED_PIN, HIGH); }
coap.sendResponse(ip, port, packet.messageid, messageBuffers);
```


RASPBERRY PI
SMALL COMPUTER
BIG POSSIBILITIES
ARDUINO R4 WIFI
SIMPLE IDE
POWERFUL CREATION
Linux Power
High Performance
More Connectivity
Great for AI & IoT
Easy to Use
Built-in WiFi
Renesas RA4M1
Perfect for Prototyping
VS
ARDUINO
R4
WIFI
IDEAS INTO REALITY
MAKE THINGS HAPPEN

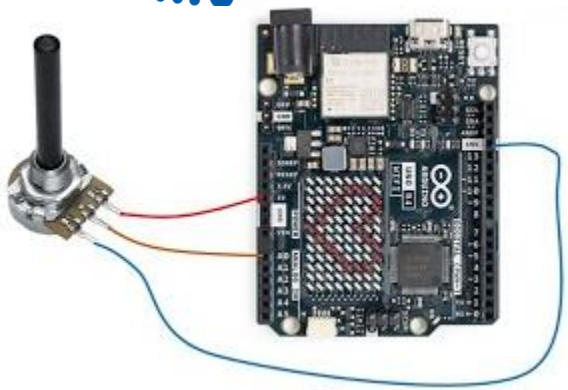
แบบฝึกหัดที่ 2. PUT Request

Coap Server





CoAP Client

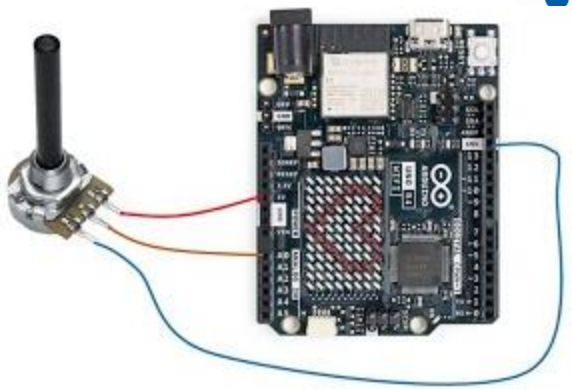


- ให้ต่อ ตัวต้านทานปรับค่าได้ ที่ CoAP Client แล้วอ่านค่า แรงดัน (0-5V)
  - โดยใช้คำสั่ง `analogRead` แล้ว PUT Request ค่าที่อ่านได้ไปยัง CoAP Server
- ให้ต่อ LED ที่ CoAP Server เมื่อได้รับค่า Payload ก็ให้ปรับความสว่างของ LED ตามค่า payload ที่ได้รับมา

Hint Raspi : - ใช้ เรื่อง PWM ปรับค่า Duty cycle เพื่อปรับความสว่าง
- โดยที่ เมื่ออ่านค่าแรงดันแล้ว ให้ Scale ให้อยู่ในช่วง 0-100

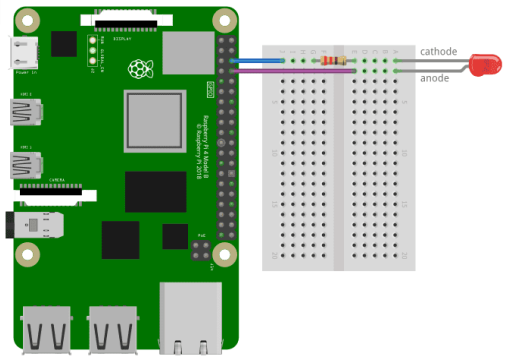
แบบฝึกหัดที่ 3. GET & Observe

Coap Server





CoAP Client



- ให้ต่อ ตัวต้านทานปรับค่าได้ ที่ CoAP Server แล้วอ่านค่า แรงดัน (0-5V) สำหรับรองรับ GET
  - ใช้คำสั่ง `analogRead`
- ลองให้ CoAP Client ทำการ GET Request ค่าที่ได้ จากการหมุนตัวต้านทานปรับค่าได้
- ค้นคว้าและเขียนโปรแกรมให้ CoAP Server รองรับ Observe Feature
- ให้ต่อ LED ที่ CoAP Client และ เขียนโปรแกรมให้รองรับการรับข้อมูลแบบ Observe
- เมื่อหมุนความต้านทานปรับค่าได้ ที่ CoAP Server ให้ LED ที่ CoAP Client มีความสว่างตามค่าที่ปรับ

Hint Raspi : - ใช้ เรื่อง PWM ปรับค่า Duty cycle เพื่อปรับความสว่าง
- โดยที่ เมื่ออ่านค่าแรงดันแล้ว ให้ Scale ให้อยู่ในช่วง 0-100


THE
END